

PRODUCTION INTEGRATION GUIDE

# Companion Profile Engine

## 机器人公司生产 API 完整接入指南

| 交付项  | 内容                           |
|------|------------------------------|
| 接口版本 | v0.2.0                       |
| 指南日期 | 2026-08-07                   |
| 运行状态 | 已上线；HTTPS、应用和 PostgreSQL 正常  |
| 接入模式 | 机器人公司服务端通过 HTTPS 调用，无需部署我方源码 |

**交付结论** 该生产 API 已完成真实线上验收（10/10 通过）。负责人只需按本文配置服务端凭据并依次完成健康检查、用户初始化和消息摄取，即可投入联调。

**机密提醒** 本文含真实生产 API Key。只能发给机器人公司负责接入的服务端负责人；请勿发群、勿放前端、勿提交代码仓库、勿写入日志。若误发，请立即联系交付方轮换密钥。

### 1. 负责人实际需要做什么

1. 在机器人公司后端的密钥管理系统中保存 BASE\_URL、TENANT\_ID、API\_KEY。
2. 确认后端服务器可出站访问 companion-profile-engine.zeabur.app 的 HTTPS 443 端口；不需要开放入站端口。
3. 按第 3 节执行健康检查和鉴权检查。
4. 按第 5 节实现“读取/初始化 → 摄取消息 → 使用 reply\_hints 生成回复”的主链路。
5. 按第 11 节完成上线验收。

除本文外，不需要额外安装包、源码、数据库账号或云平台账号。OpenAPI 文档可以在线查看，但正式集成以本文约定为准。

## 2. 生产环境地址与正式凭据

| 变量           | 正式值                                                               |
|--------------|-------------------------------------------------------------------|
| BASE_URL     | https://companion-profile-engine.zeabur.app                       |
| TENANT_ID    | robot-company-prod                                                |
| API_KEY      | 8bce7252cf3784197d654f81c36442d385bd9a40343e7f11adfccd99acc9b35be |
| Swagger UI   | https://companion-profile-engine.zeabur.app/docs                  |
| OpenAPI JSON | https://companion-profile-engine.zeabur.app/openapi.json          |

建议配置为后端环境变量：

```
PROFILE_ENGINE_BASE_URL=https://companion-profile-engine.zeabur.app
PROFILE_ENGINE_TENANT_ID=robot-company-prod
PROFILE_ENGINE_API_KEY=8bce7252cf3784197d654f81c36442d385bd9a40343e7f11adfccd99acc9b35be
```

**凭据规则** API Key 与租户 ID 是一组固定凭据。不得只凭 API Key 允许调用任意租户；所有请求必须同时发送这两个 Header。

## 3. 五分钟连通性验证

### 3.1 健康检查（无需鉴权）

```
curl.exe -sS "https://companion-profile-engine.zeabur.app/health"
```

必须得到 HTTP 200，且 JSON 同时满足 status=ok、services.application=ok、services.database=ok：

```
{
  "status": "ok",
  "service": "companion-profile-engine",
  "version": "0.2.0",
  "services": {"application": "ok", "database": "ok"}
}
```

### 3.2 鉴权与画像读取

```
curl.exe -sS "https://companion-profile-engine.zeabur.app/v1/profiles/connectivity-check" ^
-H "X-Tenant-ID: robot-company-prod" ^
-H "X-API-Key: 8bce7252cf3784197d654f81c36442d385bd9a40343e7f11adfccd99acc9b35be"
```

若 connectivity-check 尚未初始化，得到 HTTP 404 是正确结果，表示域名、TLS、鉴权和数据库查询链路均已打通。得到 401 表示 Header 或密钥有误。

### 3.3 Windows PowerShell 一次性检查

```
$base = "https://companion-profile-engine.zeabur.app"
$tenant = "robot-company-prod"
$key = "8bce7252cf3784197d654f81c36442d385bd9a40343e7f11adfc99acc9b35be"
$headers = @{"X-Tenant-ID"=$tenant; "X-API-Key"=$key}
Invoke-RestMethod "$base/health"
try { Invoke-RestMethod "$base/v1/profiles/connectivity-check" -Headers $headers }
catch { $_.Exception.Response.StatusCode.value__ } # 首次应为 404
```

## 4. HTTP 约定

| 项目   | 要求                                                           |
|------|--------------------------------------------------------------|
| 传输   | HTTPS; JSON; UTF-8                                           |
| 鉴权   | 所有 /v1 请求带 X-Tenant-ID 与 X-API-Key                           |
| 写请求  | 所有 POST 额外带 Idempotency-Key 与 Content-Type: application/json |
| 用户标识 | 租户内稳定唯一且不可复用; 不要直接用手机号、昵称或会话 ID                              |
| 版本控制 | 写请求使用最近一次成功响应的 profile_version                               |
| 请求追踪 | 记录响应头 X-Request-ID; 报障时提供, 禁止记录 API Key                      |
| 超时建议 | 连接 5 秒、总请求 30 秒; 网络错误可按幂等规则有限重试                              |

### 4.1 必带 Header

```
X-Tenant-ID: robot-company-prod
X-API-Key: 8bce7252cf3784197d654f81c36442d385bd9a40343e7f11adfc99acc9b35be
Content-Type: application/json # POST 请求
Idempotency-Key: <稳定且唯一的业务操作 ID> # POST 请求
```

### 4.2 幂等键规则

- 6. 同一个业务操作因超时而重试时, Idempotency-Key 和请求体必须完全相同。
- 7. 不同接口、不同用户、不同消息不得复用同一个幂等键。
- 8. 推荐: 初始化用 init:<USER\_ID>; 消息摄取直接使用机器人侧的 message\_id。
- 9. 同一幂等键配不同请求体会返回 422; 此时不要继续盲目重试。

## 5. 必须实现的主调用流程

**推荐顺序** 收到用户消息 → 读取或确认画像版本 → 必要时初始化 → 调用 messages:ingest → 把 reply\_hints 注入机器人回答策略 → 保存新 profile\_version → 调用聊天模型生成回复。

### 5.1 读取已有画像

```
curl -sS "https://companion-profile-engine.zeabur.app/v1/profiles/$USER_ID" \
-H "X-Tenant-ID: robot-company-prod" \
-H "X-API-Key: 8bce7252cf3784197d654f81c36442d385bd9a40343e7f11adfc99acc9b35be"
```

10. HTTP 200: 读取并保存 profile\_version。

11. HTTP 404: 该用户还没有画像, 执行 5.2 初始化。

### 5.2 首次初始化

```
curl -sS -X POST "https://companion-profile-engine.zeabur.app/v1/profiles:init" \
-H "X-Tenant-ID: robot-company-prod" \
-H "X-API-Key: 8bce7252cf3784197d654f81c36442d385bd9a40343e7f11adfc99acc9b35be" \
-H "Idempotency-Key: init:robot-user-001" \
-H "Content-Type: application/json" \
-d '{
  "tenant_user_id": "robot-user-001",
  "display_name": "用户昵称",
  "consent": {
    "profile": true,
    "sensitive_inference": false
  }
}'
```

初始化成功返回 profile\_version=1。只有用户明确同意画像处理后, consent.profile 才能为 true。

生日推断或九型数据属于敏感推断, 只有取得相应授权后才能把 sensitive\_inference 设为 true。

可选初始化字段: birth\_date (YYYY-MM-DD)、birth\_time、timezone、enneagram。没有可靠信息时不要传空字符串或猜测值。

### 5.3 每轮摄取用户消息

```
curl -sS -X POST "https://companion-profile-engine.zeabur.app/v1/profiles/robot-user-001/messages:ingest" \
-H "X-Tenant-ID: robot-company-prod" \
-H "X-API-Key: 8bce7252cf3784197d654f81c36442d385bd9a40343e7f11adfc99acc9b35be" \
-H "Idempotency-Key: turn-001" \
-H "Content-Type: application/json" \
-d '{
  "conversation_id": "session-001",
  "message_id": "turn-001",
  "expected_profile_version": 1,
  "occurred_at": "2026-08-07T08:00:00Z",
  "text": "以后回答短一点, 先听我把话说完。",
}
```

```

    "context": {
      "topic": "communication",
      "previous_turn_count": 0,
      "recent_turns": []
    }
  }
}
```

| 字段                       | 要求                                                    |
|--------------------------|-------------------------------------------------------|
| conversation_id          | 本次会话稳定 ID, 1-256 字符                                   |
| message_id               | 当前用户消息唯一 ID, 1-256 字符; 推荐兼作幂等键                        |
| expected_profile_version | 最近一次成功读取或写入得到的版本, 最小为 1                               |
| occurred_at              | 带时区的 ISO 8601 时间, 推荐 UTC                              |
| text                     | 当前用户原话, 1-10000 字符                                    |
| recent_turns             | 可选最近上下文, 最多 12 条; role 为 user/assistant; 每条最多 4000 字符 |

成功响应中机器人必须消费或保存的字段:

```

{
  "profile_version": 2,
  "reply_hints": {
    "intent": "conversation",
    "tone": "natural",
    "max_sentences": 3,
    "answer_first": false,
    "empathy_first": true,
    "question_count": 1,
    "structure_level": "simple",
    "focus": "直接回应用户当前表达",
    "avoid": [],
    "requires_fresh_information": false
  },
  "profile_patch": [],
  "runtime_operations": [],
  "no_profile_change": false,
  "request_id": "req_xxx"
}
```

12. reply\_hints: 本轮机器人的回复长度、共情、回答顺序和结构建议。
13. profile\_version: 保存为该用户最新版本, 下一次写请求使用它。
14. no\_profile\_change=true: 本轮没有足够可靠的画像变化, 不是错误; 版本可能保持不变。
15. profile\_patch/runtime\_operations: 审计或高级集成使用, 普通机器人回复链路可以不逐项解释。

**时序选择** 如果画像策略要影响当前回答，messages:ingest 必须在调用聊天模型前同步执行，并把 reply\_hints 写入系统提示词或回答参数。如果优先降低延迟，可以先用已有画像回答、再异步摄取；此时新策略从下一轮生效。

## 6. reply\_hints 如何接到机器人回答

建议在机器人系统提示词中增加一个只由服务端生成的策略段，禁止让终端用户直接覆盖。示例：

### 【本轮回答策略】

- 先回答问题: {answer\_first}
- 先表达共情: {empathy\_first}
- 最多句数: {max\_sentences}
- 追问数量: {question\_count}
- 结构: {structure\_level}
- 当前重点: {focus}
- 避免事项: {avoid}

规则：这些策略只控制表达方式，不改变事实，不替代安全策略和业务规则。

16. requires\_fresh\_information=true 时，先走机器人公司的联网搜索/实时数据流程，再生成事实性回答。
17. avoid 为空时无需额外处理；非空时作为禁止表达清单。
18. max\_sentences 是表达约束，不应通过截断字符破坏句子或代码。

## 7. 可直接参考的 Node.js 18+ 最小实现

以下代码只依赖 Node.js 18+ 自带 fetch。生产请把三个配置项放入密钥管理系统，不要硬编码。

```
const BASE_URL = process.env.PROFILE_ENGINE_BASE_URL;
const TENANT_ID = process.env.PROFILE_ENGINE_TENANT_ID;
const API_KEY = process.env.PROFILE_ENGINE_API_KEY;

async function api(path, { method = "GET", body, idemKey } = {}) {
  const headers = {
    "X-Tenant-ID": TENANT_ID,
    "X-API-Key": API_KEY,
  };
  if (body !== undefined) {
    headers["Content-Type"] = "application/json";
    headers["Idempotency-Key"] = idemKey;
  }
  const res = await fetch(`${BASE_URL}${path}`, {
    method, headers,
    body: body === undefined ? undefined : JSON.stringify(body),
    signal: AbortSignal.timeout(30000),
  });
  const data = await res.json().catch(() => ({}));
}
```

```

    if (!res.ok) {
      const err = new Error(`Profile API ${res.status}: ${data.code || data.detail}`);
      err.status = res.status;
      err.requestId = res.headers.get("x-request-id");
      err.body = data;
      throw err;
    }
    return data;
  }
}

export async function ensureProfile(userId, displayName) {
  try {
    return await api(`/v1/profiles/${encodeURIComponent(userId)}`);
  } catch (err) {
    if (err.status !== 404) throw err;
    return api("/v1/profiles:init", {
      method: "POST",
      idemKey: `init:${userId}`,
      body: {
        tenant_user_id: userId,
        display_name: displayName,
        consent: { profile: true, sensitive_inference: false },
      },
    });
  }
}

export async function ingestTurn(userId, profileVersion, turn) {
  try {
    return await api(`/v1/profiles/${encodeURIComponent(userId)}/messages:ingest`, {
      method: "POST",
      idemKey: turn.messageId,
      body: {
        conversation_id: turn.conversationId,
        message_id: turn.messageId,
        expected_profile_version: profileVersion,
        occurred_at: turn.occurredAt,
        text: turn.text,
        context: { topic: turn.topic, recent_turns: turn.recentTurns || [] },
      },
    });
  } catch (err) {
    if (err.status !== 409) throw err;
    // 重新读取最新版本, 再以同一业务幂等键和同一业务内容重试一次。
    const latest = await api(`/v1/profiles/${encodeURIComponent(userId)}`);
    return api(`/v1/profiles/${encodeURIComponent(userId)}/messages:ingest`, {
      method: "POST",
      idemKey: turn.messageId,
      body: {
        conversation_id: turn.conversationId,
        message_id: turn.messageId,
        expected_profile_version: latest.profile_version,
        occurred_at: turn.occurredAt,
        text: turn.text,
        context: { topic: turn.topic, recent_turns: turn.recentTurns || [] },
      },
    });
  }
}

```

**实现注意** 409 后请求体中的 `expected_profile_version` 会变化, 因此若服务端将幂等键严格绑定完整请求体, 重试应使用新的派生幂等键 (例如 `messageId + ':retry1'`)。本服务当前以请求体摘要校验幂等键, 故推荐重新读版本后使用派生键, 避免 422。最多重试一次。

## 8. 并发、409 与重试策略

- 19. 每个用户串行处理画像写请求；不同用户可以并发。
- 20. 每次成功写入后，原子保存响应的 profile\_version。
- 21. 收到 409 时，重新 GET 当前画像，取得最新 profile\_version。
- 22. 用新的 expected\_profile\_version 和派生幂等键重试一次；仍冲突则进入队列或人工排查，禁止无限循环。
- 23. 网络超时且不知道服务是否已处理时，先用完全相同的幂等键和请求体重试；不要生成新键。

| 场景          | 是否重试 | 方式                        |
|-------------|------|---------------------------|
| 连接超时/网络中断   | 是    | 同一幂等键 + 同一请求体，最多 2 次，指数退避 |
| 409 版本冲突    | 是    | 重新 GET；新版本 + 派生幂等键，最多 1 次 |
| 401/403/422 | 否    | 修正配置、授权或请求字段              |
| 503         | 是    | 1s/2s/4s 指数退避，最多 3 次      |
| 5xx 其他      | 有限   | 记录 X-Request-ID；最多 2 次后告警 |

## 9. 可选管理接口

| 用途                 | 接口                                              |
|--------------------|-------------------------------------------------|
| 查看画像及证据依据          | GET /v1/profiles/{user_id}/explain?field=<字段路径> |
| 用户/专家明确更正          | POST /v1/profiles/{user_id}:correct             |
| 写入已授权九型测评          | POST /v1/profiles/{user_id}:set-enneagram       |
| 撤销记忆/证据/生日/九型/全部画像 | POST /v1/profiles/{user_id}:forget              |
| 查询生产规则包            | GET /v1/rule-packs/current                      |

## 9.1 用户明确更正

```
POST /v1/profiles/{user_id}:correct
{
  "expected_profile_version": 2,
  "target_path": "core_traits.energy_mode.extroversion",
  "value": 0.8,
  "reason": "用户明确更正"
}
```

## 9.2 写入九型测评

```
POST /v1/profiles/{user_id}:set-enneagram
{
  "expected_profile_version": 2,
  "enneagram": {
    "core_type": 5,
    "wing": 6,
    "primary_instinct": "SP",
    "secondary_instinct": "SO",
    "source": "external_assessment",
    "confidence": 0.9
  },
  "reason": "机器人公司已完成并取得授权的测评结果"
}
```

九型需要 sensitive\_inference=true。wing 必须是主型相邻类型；两个 instinct 不能相同；core\_type 为 1-9。

## 9.3 撤回或遗忘

```
POST /v1/profiles/{user_id}:forget
{
  "expected_profile_version": 3,
  "scope": "all_profile",
  "reason": "用户撤回画像授权"
}
```

scope 可选 memory、evidence、birth\_inference、enneagram、all\_profile。memory/evidence 必须同时提供 target\_id。all\_profile 会停止后续画像推断并清理运行时偏好、状态、记忆、数字密码和九型数据，使证据/记忆失效；它不会物理删除历史审计记录。物理删除需走双方另行约定的数据删除流程。

**禁止调用** 不要调用 POST /v1/profiles/{user\_id}:reset。该接口只用于开发测试，生产环境固定返回 404；即使在线 OpenAPI 中可见，也不属于对外生产合同。

## 10. 错误码与现场处理

| HTTP | 含义                           | 负责人处理                           |
|------|------------------------------|---------------------------------|
| 400  | 请求语法或业务格式错误                  | 检查 JSON、URL 与 Content-Type      |
| 401  | 租户或 API Key 错误               | 停止重试；核对两个 Header 与密钥管理配置        |
| 403  | 未取得画像/敏感推断授权，或<br>画像已关闭      | 不自动重试；回到用户授权流程                  |
| 404  | 用户不存在；或生产禁用的测试<br>接口         | 读取场景可初始化；reset 场景必须停止           |
| 409  | profile_version 已过期          | 重新读取最新版本后最多重试一次                 |
| 422  | Header/字段校验失败，或幂等<br>键与请求体冲突 | 修正请求；不要原样无限重试                   |
| 503  | 语义处理临时不可用                    | 有限指数退避；保留 X-Request-ID          |
| 5xx  | 服务端异常                        | 有限重试并告警；提供时间、路径、状态、X-Request-ID |

标准错误体示例：

```
{
  "request_id": "req_xxx",
  "code": "profile_version_conflict",
  "message": "画像版本不匹配",
  "details": {
    "expected_profile_version": 1,
    "actual_profile_version": 2
  }
}
```

FastAPI Header/字段校验错误可能使用 detail 数组，而不是上述 code 结构。客户端应以 HTTP 状态为主，并兼容读取 code、detail、message。

## 11. 安全、隐私与数据边界

- 24. API 只能由机器人公司服务端调用；客户端、网页、小程序或机器人终端不得持有生产 Key。
- 25. 密钥应放入 Secret Manager/KMS 或受控环境变量；日志、告警、埋点、错误堆栈均需脱敏。
- 26. tenant\_user\_id 使用内部不可逆 ID；不要直接传手机号、身份证、邮箱或昵称作为主键。

- 27. 只发送实现画像和回复策略所必需的消息上下文；recent\_turns 保持最少必要。
- 28. 未取得 profile 授权不得初始化；未取得 sensitive\_inference 授权不得提交生日推断或九型数据。
- 29. 当前生产语义提取器为本服务内置确定性规则，不把用户原话发送给外部语义模型。
- 30. 如密钥疑似泄露，立即停止使用并联系交付方轮换；不要把泄露的 Key 再粘贴到普通聊天或工单。

## 12. 上线验收清单

| 验收项    | 通过标准                                             |
|--------|--------------------------------------------------|
| HTTPS  | 无需忽略证书校验即可访问正式域名                                 |
| 健康检查   | HTTP 200; status=ok; application=ok; database=ok |
| 错误 Key | 返回 401                                           |
| 租户隔离   | 正确 Key + 错误租户返回 401                              |
| 初始化    | 新用户返回 profile_version=1                          |
| 读取     | 能按同一 tenant_user_id 读取画像                         |
| 消息摄取   | 返回 reply_hints 与 profile_version                 |
| 幂等     | 相同幂等键 + 相同请求体重复提交结果一致                            |
| 并发版本   | 旧 profile_version 写入返回 409                       |
| 生产边界   | reset 返回 404                                     |

**我方已验收** 2026-08-07 已使用本文正式域名、租户与密钥完成线上 10/10 验收：健康、数据库、错误密钥、租户隔离、初始化、读取、消息摄取、幂等、409、生产禁用 reset 全部通过。

## 13. 联调报障信息模板

出现问题时，只提供以下内容；不要提供 API Key、完整用户原话或完整画像：

```
环境: production
发生时间: YYYY-MM-DD HH:mm:ss (注明时区)
请求方法与路径: POST /v1/...
HTTP 状态: xxx
X-Request-ID: ...
tenant_user_id: 建议仅提供末 6 位或内部可追踪的脱敏值
是否为重试: 是/否
期望结果: ...
实际结果: ...
已确认: 健康检查结果 / 出站网络 / Header 是否存在
```

## 14. 最终交接说明

机器人公司负责人完成以下五件事即可正式跑通:

31. 安全保存第 2 节的三个生产变量。
32. 完成第 3 节连通性验证。
33. 实现第 5 节主链路, 并按用户维度保存 profile\_version。
34. 把 reply\_hints 注入本轮机器人回答策略。
35. 按第 12 节逐项验收; 通过后上线。

**唯一交付文件** 本指南已经包含运行地址、真实凭据、完整调用顺序、可复制命令、Node.js 示例、错误处理、安全边界和验收标准。接入负责人无需再索取其他文件即可开始联调。